



Pontificia Universidad Católica de Valparaíso
Facultad de Ingeniería
Escuela de Ingeniería Informática

Telemedicina

Autor: Francisca Zamora

Asignatura: Ingeniería web y móvil

Profesor: Gabriel Toro

Introducción

El presente proyecto surge en respuesta a la creciente necesidad de modernizar los canales de atención de salud mediante herramientas digitales avanzadas. La telemedicina no solo representa una alternativa a la consulta presencial, sino que se posiciona como una infraestructura crítica para el seguimiento continuo de pacientes.

Se desarrolló una aplicación multiplataforma con Ionic y React que permite a médicos y pacientes intercambiar información clínica en tiempo real: registro de signos vitales, chat directo, videollamadas, recetas médicas e indicaciones personalizadas, todo respaldado por una base de datos MySQL y una API REST con Node.js.

El objetivo principal es transformar la relación médico-paciente de un modelo reactivo (atención solo ante la crisis) a un modelo proactivo y preventivo, centrado en el monitoreo sistemático y la comunicación eficiente.

Entrega parcial 1:

1. Justificación del problema y análisis del usuario objetivo. (EP 1.2)

Planteamiento del Problema

La problemática central identificada es la discontinuidad de la información médica una vez que el paciente abandona el centro asistencial o bien termina su videollamada. Actualmente, los pacientes con enfermedades crónicas enfrentan dificultades para registrar cambios diarios en su condición, ya que si se encuentran mal o no tienen bien claro si mejoraron, deben ir a una consulta presencial, pero esto genera diagnósticos basados en datos incompletos y subjetivos. Esta falta de visibilidad clínica entre consultas impide que el profesional de la salud pueda intervenir de manera oportuna ante variaciones críticas de signos vitales, derivando en complicaciones que frecuentemente terminan en servicios de urgencia saturados. Además, se busca que la conectividad entre médico-paciente sea más sencilla a través de la casa.

Justificación del problema

La implementación de esta solución tecnológica se justifica por su capacidad de centralizar el flujo de datos clínicos en un entorno seguro y accesible. El paciente debe esperar entre controles médicos para comunicar sus síntomas, período durante el cual sus signos vitales pueden variar significativamente sin que el médico tenga visibilidad de ello.

La solución propuesta aborda directamente esta brecha mediante tres mecanismos: registro diario de signos vitales desde el hogar, canal de comunicación asíncrono médico-paciente, y sistema de alertas automáticas ante parámetros críticos. Al integrar herramientas de autoregistro de signos vitales y un canal de comunicación directo, la plataforma permite:

- **Optimización de Tiempos:** Reduce la necesidad de traslados para consultas que pueden resolverse de forma remota.
- **Toma de Decisiones Basada en Datos:** Proporciona al médico un historial gráfico y detallado de la evolución del paciente.
- **Prevención y Alerta:** Permite la detección temprana de riesgos mediante un sistema de semaforización visual según la gravedad de los registros.

Análisis del usuario objetivo

La plataforma ha sido diseñada bajo principios de **Diseño Centrado en el Usuario (UCD)**, reconociendo dos perfiles con necesidades técnicas diferenciadas:

- **Perfil Paciente:** Personas que requieren monitoreo constante, con una interfaz simplificada para facilitar el registro de datos vitales, incluso en adultos mayores. Registra su presión arterial diaria y comunica síntomas sin desplazarse. Si olvida síntomas al llegar a la consulta, el médico tiene datos anotados. Hay una reducción del tiempo entre detección de síntoma y respuesta médica de días a horas.

- **Perfil Médico:** Profesionales que necesitan gestionar múltiples pacientes, visualizar datos históricos y priorizar casos críticos mediante alertas automáticas. Hace seguimiento crónico a pacientes que requieren control frecuente. Priorizar atención según gravedad real, no por orden de llegada.

2. Definición de Requerimientos (EP 1.1)

Requerimientos Funcionales

Para esta etapa, se han definido y diseñado las interfaces para los siguientes 9 requerimientos funcionales:

RF1: Registro de Estado Diario

- **Rol:** Paciente
- **Criterio de Aceptación Medible:** El paciente puede ingresar presión arterial, pulso, temperatura, nivel de dolor y síntomas desde un formulario. El sistema guarda el registro en MySQL con timestamp y responde HTTP 201 en menos de 500ms.

RF2: Visualización de historial de salud

- **Rol:** Paciente
- **Criterio de Aceptación Medible:** El dashboard muestra el último registro clínico actualizado inmediatamente tras cada envío, diferenciando estado Estable (verde) o Crítico (rojo).

RF3: Gestión de Agenda y Citas

- **Rol:** Paciente / Médico
- **Criterio de Aceptación Medible:** El sistema lista citas futuras y pasadas filtradas por fecha. El paciente puede agendar seleccionando médico, fecha y hora. El médico visualiza todas las citas de sus pacientes.

RF4: Comunicación médico-paciente

- **Rol:** Paciente / Médico
- **Criterio de Aceptación Medible:** El chat permite envío y recepción de mensajes en tiempo real con polling cada 10 segundos. Los mensajes se persisten en MySQL y el historial es visible para ambos roles.

RF5: Gestión de pacientes

- **Rol:** Médico
- **Criterio de Aceptación Medible:** El médico visualiza lista completa de pacientes asignados con nombre, RUT y estado de salud. Puede buscar por nombre o RUT y acceder a la ficha clínica individual en menos de 1 segundo.

RF6: Sistema de Alertas Visuales y por Email

- **Rol:** Médico / Sistema
- **Criterio de Aceptación Medible:** El sistema clasifica automáticamente el estado como Crítico si temperatura $\geq 38^{\circ}\text{C}$, pulso ≥ 100 bpm o dolor $\geq 8/10$. Envía email automático al médico asignado vía Nodemailer en menos de 5 segundos tras el registro.

RF7: Ficha Clínica del Paciente

- **Rol:** Médico
- **Criterio de Aceptación Medible:** La ficha muestra nombre, RUT, correo, región, comuna, último registro clínico con signos vitales e historial completo de registros anteriores, todos obtenidos desde MySQL en tiempo real.

RF8: Emisión de recetas médicas

- **Rol:** Médico
- **Criterio de Aceptación Medible:** El médico puede crear recetas seleccionando paciente, medicamentos e indicaciones. Las recetas se guardan en MySQL y son visibles para el paciente en /recetas. El médico puede eliminarlas con confirmación.

RF9: Registro de cumplimiento de tratamiento

- **Rol:** Paciente
- **Criterio de Aceptación Medible:** El paciente puede indicar SI o NO cumplió su tratamiento diario con comentario opcional. El historial de cumplimiento se persiste en localStorage y MySQL, visible en /seguimiento.

RF10: Registro y Validación de Usuarios

- **Rol:** Paciente / Médico
- **Criterio de Aceptación Medible:** El formulario valida campos obligatorios (nombre, RUT, correo, región, comuna, contraseña) con indicadores visuales rojos. Las contraseñas se almacenan con hash bcrypt. El sistema asigna automáticamente un médico al nuevo paciente.

RF11: Videollamada de Telemedicina

- **Rol:** Paciente / Médico
- **Criterio de Aceptación Medible:** El paciente visualiza sus citas programadas y puede unirse a la sala de videollamada. La interfaz incluye controles de micrófono, cámara y botón de colgar.

Requerimientos No Funcionales (RNF)

RNF1: Seguridad:

El sistema implementa autenticación JWT con expiración de 2 horas, hash de contraseñas con bcrypt, middleware verificarToken en rutas protegidas del backend, Helmet.js para cabeceras HTTP seguras, rate limiting de 100 requests/15min general y 10 requests/15min en login, y CORS restringido a los puertos del frontend.

Métrica de cumplimiento: Un intento de acceso sin token válido debe retornar HTTP 401 en menos de 50ms. Tras 10 intentos fallidos de login desde la misma IP, el sistema debe bloquear la IP por 15 minutos.

RNF2 : Rendimiento:

El tiempo de respuesta de los endpoints de la API REST no debe superar 500ms bajo condiciones normales de uso local. La navegación entre vistas del frontend no debe generar recargas de página completa gracias a la arquitectura SPA de React Router.

Métrica de cumplimiento: Las pruebas realizadas en Postman muestran tiempos de respuesta entre 13ms y 489ms para todos los endpoints implementados. La navegación entre pantallas ocurre en menos de 300ms al ser renderizado del lado del cliente.

RNF3:Usabilidad:

La interfaz diferencia visualmente los roles mediante menús laterales distintos. Los estados críticos se identifican con color rojo y los estables con verde en todas las vistas relevantes. Los formularios muestran validaciones visuales inmediatas con bordes rojos antes de permitir el envío.

Métrica de cumplimiento: Un usuario nuevo debe poder registrar su estado de salud en menos de 3 clics desde el dashboard. La semaforización visual permite identificar el estado de un paciente sin necesidad de leer texto adicional.

RNF4: Escalabilidad:

La arquitectura modular del backend permite agregar nuevos módulos sin modificar el código existente. Los índices implementados en MySQL garantizan que el tiempo de consulta no aumente proporcionalmente con el volumen de registros.

Métrica de cumplimiento: Con índices aplicados, una consulta de historial clínico por usuarioId debe ejecutarse en tiempo $O(\log n)$ independiente del número total de registros en la tabla.

3. Bocetos de UI/UX y Prototipo en Figma (EP 1.3)

Se han desarrollado mockups funcionales que cubren el flujo completo de la aplicación, diferenciando explícitamente entre las versiones **Desktop** y **Mobile**.

Actualmente, **el prototipo implementado en Figma corresponde al módulo principal de Telemedicina**, permitiendo validar navegación, arquitectura de información y experiencia de usuario del sistema.

Respecto a la integración con el portal institucional de la Municipalidad de Santo Domingo, esta fue definida a nivel conceptual como parte del diseño UX, proponiendo un **acceso directo desde el sitio oficial Municipalidad de Santo Domingo hacia el módulo de Telemedicina**. Esta integración visual **actualmente no se encuentra prototipada en Figma**, ya que se desarrolló los mockups para entrar a los módulos correspondientes

Los mockups se encuentran en la carpeta otros, en carpeta zip descargado.

Formularios de Acceso y Validación Visual

Se diseñaron dos formularios críticos con un enfoque en **Diseño Centrado en el Usuario (UCD)**:

- **Pantalla de Inicio de Sesión (Login):**
 - **Campos:** RUT y Contraseña.
 - **UX:** Se incluyen estados de validación visual, bordes rojos y mensajes de "Dato Requerido" para prevenir errores de envío. El diseño es limpio, con el logo institucional en alta jerarquía para generar confianza.
- **Pantalla de Registro:**
 - **Campos requeridos:** Nombre y Apellido, RUT, Correo Electrónico, Región, Comuna, Contraseña y Confirmación de Contraseña.
 - **Validaciones:** Incluye el checkbox obligatorio de **Aceptación de Términos y Condiciones**. El diseño utiliza una distribución clara para evitar la fatiga visual ante la cantidad de campos.

Pantallas Funcionales (Mockups)

Se diseñaron las siguientes pantallas diferenciadas, cada una asociada a un requerimiento funcional:

1. **Dashboard del Paciente:** Presenta un resumen de salud inmediato, destacando el último estado registrado, las próximas citas médicas y mensaje enviado por médico. Al igual que tiene iconos directos para saber indicaciones, recomendaciones, agendar, etc.
2. **Registro de Estado de Salud:** Formulario intuitivo con selectores para registrar presión arterial, pulso y síntomas.
3. **Módulo de Chat (Paciente):** Interfaz de mensajería para comunicación directa con el médico asignado.
4. **Dashboard del Médico:** Panel de gestión que muestra el resumen de pacientes críticos mediante un sistema de alertas.

5. **Lista de Pacientes Asignados:** Tabla con filtros de búsqueda por nombre o RUT, facilitando la administración de la carga asistencial. Aparecen de igual forma, de los primeros los pacientes en estado crítico.
6. **Ficha Clínica Detallada:** Visualización de antecedentes del paciente y gráficos de evolución de signos vitales o enfermedades que padezca.
7. **Interfaz de Videollamada:** Pantalla con controles de audio/video y vista compartida para la atención remota.

Se utilizó una paleta de colores **Verde Esmeralda (#00875E)** y blancos para transmitir higiene y profesionalismo médico. La jerarquía se establece mediante el peso tipográfico y la ubicación espacial, situando la información de alerta siempre en la parte superior del campo visual.

4: Definición de Arquitectura de Navegación y Experiencia del Usuario (EP 1.4)

Se detalla la organización estructural y el flujo de interacción de la plataforma, garantizando una experiencia de usuario coherente.

(a) Rutas Principales y Secundarias

La aplicación implementa un sistema de enrutamiento basado en **React Router**, dividiendo el acceso en dos niveles jerárquicos:

- **Rutas Principales (Nivel 1):** Son los puntos de acceso públicos.
 - **Login:** Interfaz de autenticación donde el usuario ingresa su RUT y contraseña. El sistema bloquea el acceso si los campos están vacíos mediante validaciones de estado.
 - **Registro:** Formulario de captura de datos, Nombre, RUT, Correo, Región, Comuna y Contraseña necesario para nuevos usuarios.
- **Rutas Secundarias (Nivel 2 Protegidas):** Vistas operativas accesibles únicamente tras una autenticación exitosa.
 - **Home:** Dashboard centralizado.
 - **Chat:** Módulo de comunicación con el especialista.
 - **Registro-estado:** Formulario de entrada de signos vitales.

(b) Relaciones Jerárquicas entre Vistas

La jerarquía se organiza de forma arbórea desde un nodo raíz (App.tsx).

- **Nivel Superior:** Control de sesión. Si el authService confirma la sesión, se habilita el IonRouterOutlet.
- **Nivel Intermedio:** El **Dashboard** actúa como el eje central. Todas las vistas secundarias como chat y registro de salud dependen jerárquicamente del Home, permitiendo siempre el retorno a la vista principal para mantener el contexto del usuario.

(c) Flujo de Navegación entre Funcionalidades

La interacción entre pantallas ha sido diseñada para ser fluida y sin recargas de página (SPA).

- **Desde el Dashboard:** El usuario puede navegar radialmente hacia el registro de salud o la mensajería.
- **Interacción de Datos:** Al completar un registro de salud, el flujo redirige al usuario de vuelta al Dashboard, donde se actualiza el **Resumen de Salud** para reflejar los nuevos datos ingresados, cerrando el ciclo de retroalimentación.

(d) Diferenciación de Acceso según Roles

La arquitectura de la aplicación reconoce y adapta la interfaz según el perfil del usuario:

- **Rol Paciente:** Enfocado en el autoregistro. Su navegación permite ver indicaciones médicas, exámenes y agendar citas. El menú lateral prioriza herramientas de seguimiento personal.
- **Rol Médico:** Su navegación es de **supervisión y gestión**. El médico no registra datos propios, sino que accede a una **Lista de Pacientes Asignados**. Puede entrar en la ficha de cada paciente, revisar gráficos de evolución histórica, emitir recomendaciones y activar videollamadas de telemedicina.

(e) Flujo de Principales Tareas (Task Flow)

Se han optimizado los flujos para minimizar la cantidad de clics:

1. **Task Flow Paciente:** Login -> Home -> Botón "Registrar estado" -> Ingreso de datos -> Confirmación -> Regreso al Home con resumen actualizado.
2. **Task Flow Médico:** Login -> Lista de Pacientes -> Selección de Ficha -> Revisión de alertas rojas/verdes -> acceso de Chat o Videollamada de respuesta.

(f) Puntos Críticos de Interacción

Se identificaron y resolvieron nodos de fricción para mejorar la UX:

- **Validación Bloqueante:** En Login y Registro, el botón de acción no se procesa si faltan datos obligatorios, marcando los campos en rojo para una corrección inmediata.
- **Feedback de Estado:** El uso de semaforización visual en el Dashboard permite al usuario entender su estado de salud de un solo vistazo sin leer texto denso.
- **Cierre de Sesión Seguro:** Se implementó `window.location.assign` para limpiar el historial del navegador, evitando que se pueda volver atrás a una ruta protegida tras cerrar sesión.

(g) Coherencia de Experiencia entre Dispositivos

La adaptabilidad se logra mediante el sistema de grillas de Ionic:

- **Vista Web:** El menú lateral es fijo y el contenido se despliega en paneles laterales, aprovechando el espacio horizontal.

- **Vista Móvil:** El diseño se reestructura a una sola columna. Los inputs y botones se expanden para facilitar el uso táctil, manteniendo la jerarquía de la información crítica siempre en la parte superior del scroll.

(h) Justificación Técnica

La arquitectura se sustenta en tres pilares:

1. **Usabilidad y Eficiencia:** El uso de componentes nativos de Ionic asegura transiciones suaves y un diseño familiar para el usuario.
2. **Claridad Estructural:** La separación en carpetas facilita el mantenimiento del código.
3. **Escalabilidad:** Al usar **React Router**, la estructura está lista para crecer, donde se integrarán las llamadas reales a la API y la base de datos sin necesidad de reescribir la lógica de navegación.

5. Creación del proyecto en Ionic con React (EP 1.5 y 1.6)

Componentes Utilizados

Para garantizar la coherencia visual y la escalabilidad del frontend, se utilizaron componentes nativos de **Ionic Framework**:

- **IonGrid, IonRow e IonCol:** Implementados para crear una arquitectura responsiva que adapta el Dashboard de una vista extendida en escritorio a una disposición vertical en dispositivos móviles.
- **IonSelect e IonInput:** Utilizados para la captura de datos clínicos, permitiendo validaciones visuales inmediatas (bordes rojos) ante campos obligatorios vacíos.
- **IonIcon:** Integrados para mejorar la jerarquía visual y la navegabilidad intuitiva en los menús y paneles de control.

Alcance Actual del Proyecto

En esta **entrega**, el software cumple con los siguientes hitos:

1. **Navegación Estructural:** El sistema permite el tránsito fluido entre las 4 pantallas principales implementadas, Login, Registro, Home y Registro de Estado.
2. **Seguridad de Rutas:** Implementación funcional de RutaProtegida. El acceso al contenido interno está bloqueado si no existe una sesión activa.
3. **Gestión de Sesión (Mock):** El sistema simula procesos de Login y Logout, gestionando el estado de la aplicación y redirigiendo al usuario correctamente.
4. **Validación de Formularios:** Los campos de entrada verifican la presencia de datos antes de permitir el avance, proporcionando feedback visual al usuario.

Limitaciones y Trabajo Pendiente

Se reconocen las siguientes brechas técnicas que serán resueltas en siguientes entregas:

- **Autenticación y Registro Real:** Actualmente, el sistema no valida la veracidad del RUT o la contraseña contra una base de datos; solo verifica que los campos no estén vacíos. Falta implementar la lógica de backend para verificar credenciales reales.
- **Refinamiento de UI Móvil:** Aunque la estructura es responsiva, el diseño actual está optimizado primordialmente para Web. Falta ajustar dimensiones y espaciados específicos para mejorar la experiencia táctil en dispositivos móviles.
- **Interactividad del Menú Lateral:** Los iconos y enlaces del menú de navegación, Citas, Exámenes, Indicaciones, etc. Se encuentran en estado visual (mockup). Su funcionalidad y vinculación a páginas reales están pendientes.
- **Complejidad del Registro Clínico:** El formulario de "Registro de Salud" cuenta con categorías limitadas. Se ampliará el catálogo de síntomas y constantes vitales en la siguiente fase.
- **Módulo de Chat Dinámico:** La mensajería es actualmente una interfaz estática. Falta implementar la lógica para responder mensajes, visualizar historiales pasados y filtrar conversaciones por profesional de salud.
- **Persistencia de Datos:** La información es volátil o se almacena en localStorage. Se requiere la integración de una **API REST (Node.js/Python)** y una **Base de Datos Relacional** para el manejo de información persistente y segura.

Entrega parcial 2:

1. Creación del servidor en Node.js con Express (EP 2.1)

Se implementó un servidor backend utilizando **Node.js** con el framework **Express**, configurado bajo el entorno de **TypeScript**. Esta combinación fue seleccionada debido a que Express proporciona la velocidad y flexibilidad arquitectónica que requiere una aplicación médica, mientras que TypeScript añade un sistema de tipado estático que previene errores de compilación y manipulación en los datos sensibles de salud.

- **Estructura Modular:** En lugar de concentrar el código en un único archivo, el backend se estructuró de manera modular. El servidor principal se limita exclusivamente a inicializar los servicios y los middlewares, delegando la gestión de las rutas (rutas/) y el procesamiento de la lógica de negocio (controladores/) en módulos independientes y ordenados.
- **Middlewares Esenciales:** Se incorporó el middleware nativo `express.json()` para permitir que el servidor interprete de forma automática los cuerpos de las peticiones en formato JSON provenientes del cliente. Asimismo, se configuró el middleware `cors()` con el objetivo de otorgar permisos explícitos al frontend móvil para comunicarse e intercambiar recursos de manera segura con la API REST.

2. Configuración y Modelado de la Base de Datos Relacional (EP 2.2)

Para este proyecto se implementó una base de datos relacional con **MySQL** corriendo de forma local en **MySQL Workbench**, utilizando **Prisma ORM** como herramienta de abstracción para gestionar todo el modelo directamente desde el código fuente. Con este enfoque, el sistema opera de manera autónoma y local en el computador del evaluador sin depender de configuraciones externas.

En el archivo `schema.prisma` se definieron dos relaciones clave para estructurar los datos y asegurar que la información no quede duplicada ni huérfana:

- **Relación de Entidad Única (Médico - Paciente):** Se centralizó a todos los usuarios en la entidad `Usuario`, quedando diferenciados únicamente por el campo `rol` (médico o paciente). Un usuario con rol de médico puede tener múltiples pacientes a su cargo mediante el campo `medicoid`. Para esta relación se aplicó la regla `onDelete: SetNull`. Esto garantiza que, si un médico es desvinculado o eliminado del sistema, las cuentas de sus pacientes permanezcan activas e intactas en MySQL.
- **Relación Uno a Muchos (Paciente - Registro Clínico):** Cada ficha de signos vitales guardada en la tabla `RegistroSalud` se vincula con el ID único del paciente a través del campo `usuarioid`. En este caso se configuró la regla `onDelete: Cascade`. Esto significa

que, si un paciente es eliminado permanentemente de la plataforma, todas sus fichas clínicas asociadas se borrarán de forma automática y simultánea en Workbench, evitando registros huérfanos y resguardando la confidencialidad.

La creación física de las tablas no requirió la escritura manual de scripts SQL en Workbench. El proceso se delegó por completo al flujo del ORM mediante el comando `npx prisma db push`. Este comando interpreta las restricciones del esquema, generando de manera automática las tablas, llaves foráneas e integridades dentro de la instancia local de MySQL Workbench.

3.Desarrollo de API REST con Endpoints Básicos (EP 2.3)

Se construyó una API REST estructurada para la comunicación entre el cliente móvil y el servidor. Cada petición enviada por la aplicación recibe una respuesta bajo la arquitectura JSON, acompañada de un código de estado HTTP estándar que determina el éxito o fallo de la operación:

- **POST /api/registro:** Procesa la creación de cuentas de usuarios en el sistema. El controlador valida la presencia de los campos obligatorios y verifica que el RUT o el correo electrónico no existan previamente en la base de datos MySQL, retornando un estado 201 Created en caso de éxito.
- **POST /api/login:** Valida las credenciales de acceso mediante el RUT y la contraseña. Si los datos coinciden con el registro de la base de datos, el servidor genera la sesión correspondiente y responde con un estado 200 OK.
- **POST /api/registro-salud:** Recibe los datos biométricos ingresados (presión arterial, pulso, temperatura, nivel de dolor y síntomas). El backend ejecuta una evaluación condicional de los parámetros para determinar de forma matemática si el estado es "Estable" o "Crítico", persistiendo la ficha en MySQL con un estado 201 Created.
- **GET /api/registro-salud/:usuarioid:** Consulta las tablas de la base de datos utilizando el identificador único del paciente para extraer de forma cronológica descendente todo su historial médico y aislar el último control clínico, devolviendo un estado 200 OK.
- **GET /api/medico/:id/dashboard:** Recupera la lista de los pacientes vinculados directamente al ID del médico consultado. El endpoint consolida la información necesaria para el monitoreo y devuelve los registros con un estado 200 OK.
- **PUT / DELETE /api/registro-salud/:id:** Permiten la modificación de las observaciones clínicas o la remoción física de un registro médico específico mediante su identificador único, respondiendo con un estado 200 OK para asegurar el cumplimiento del estándar CRUD en la arquitectura REST.

4.Consumo de la API REST desde Ionic (EP 2.4)

El consumo de los servicios web de la API REST se centralizó en el frontend mediante una instancia configurada de **Axios** ubicada en src/api.ts. Esta implementación reemplazó las llamadas HTTP manuales por un cliente unificado que integra un **Interceptor de Peticiones**.

El interceptor evalúa automáticamente cada solicitud saliente dirigida hacia el servidor Express. Si detecta la existencia de una clave válida de autenticación almacenada en el navegador, extrae dicho elemento y lo acopla dinámicamente en la cabecera Authorization de la petición HTTP bajo el esquema estándar Bearer. Esta configuración técnica automatiza la transferencia de las credenciales de sesión en segundo plano, garantizando que el backend reciba y valide la identidad del usuario en cada endpoint protegido sin requerir interrupciones manuales en la interfaz de la aplicación.

5. Implementación de Autenticación con JWT (EP 2.5)

La gestión y el control de las sesiones de usuario se estructuraron mediante el estándar **JWT**. Al procesar una autenticación exitosa en el endpoint de inicio de sesión, el backend genera y firma un token criptográfico utilizando una clave secreta definida en las variables de entorno

- **Rutas Protegidas:** En el archivo de enrutamiento del frontend (App.tsx), se implementó el componente especializado <RutaProtegida>. Este componente actúa como un filtro de acceso en el cliente que evalúa la presencia del token antes de renderizar la vista solicitada. Si se intenta ingresar de forma directa a través de la URL a los módulos internos (como /home o /medico-dashboard) sin una sesión activa, el sistema intercepta la navegación y redirige de forma automática al usuario hacia la interfaz de /login.
- **Diferenciación por Roles:** El token emitido por el servidor almacena internamente en su estructura de datos (payload) el rol específico asignado al usuario en la base de datos MySQL. Al resolverse la petición de inicio de sesión, el frontend decodifica dicha información para ejecutar una bifurcación lógica en la navegación: si el registro posee el rol de médico, la aplicación realiza un direccionamiento inmediato hacia la ruta /medico-dashboard, mientras que si el rol corresponde al de un paciente, el flujo deriva el acceso hacia la interfaz de /home

6. Validación de Usuarios y Seguridad Avanzada (EP 2.6)

El sistema gestiona la seguridad y consistencia de los accesos mediante políticas de verificación en el servidor y el uso de componentes de abstracción de datos:

- **Manejo Seguro de Credenciales en Tránsito:** Durante el proceso de autenticación e inicio de sesión, el sistema valida la identidad contrastando el RUT y la contraseña en texto plano de manera directa contra los registros existentes en la base de datos MySQL. Esto permite una verificación lineal e inmediata de los datos introducidos por el usuario en la interfaz del cliente móvil antes de conceder el token de acceso.

- **Protección contra Inyección SQL mediante Parametrización Nativa:** Mediante el uso de **Prisma ORM**, todas las variables capturadas desde las entradas de texto del formulario e inyectadas en los métodos del modelo (findFirst, create, findMany) son automáticamente sanitizadas y parametrizadas por el motor del ORM antes de interactuar con MySQL. Esta capa técnica anula el riesgo de ataques por inyección SQL, debido a que las entradas del usuario son tratadas estrictamente como valores de datos y nunca como instrucciones de código SQL ejecutable.
- **Validación de Inputs en el Backend:** Los controladores de Express implementan estructuras de control condicionales que verifican la presencia y el formato correcto de los campos obligatorios (como el RUT, el correo electrónico y la contraseña) antes de dar paso a cualquier operación de escritura o lectura en MySQL, devolviendo códigos de error 400 Bad Request o 401 Unauthorized en caso de detectar datos nulos, incompletos o inconsistentes.

Limitaciones: Siguen algunas limitaciones comentadas en la entrega 1. Aunque el backend y la base de datos MySQL funcionan al 100% a través de Postman, la aplicación en Ionic todavía tiene las siguientes limitaciones técnicas que se resolverán en la entrega final:

- **Datos congelados en el inicio:** La pantalla del paciente solo muestra la información que se cargó al momento de hacer el login (/login). Si se registra un nuevo dato, la pantalla no se entera porque sigue leyendo variables fijas del localStorage.
- **Formularios desconectados:** La pantalla para registrar los signos vitales (/registro-estado) muestra los campos y guarda los datos en el backend, pero el botón de envío aún no está amarrado para refrescar la pantalla principal automáticamente. Por ahora, toda la inserción real de datos clínicos se realiza y valida de manera impecable desde Postman.
- **Pantallas en estado de maqueta:** Secciones como el chat, la agenda de citas y el historial de videollamadas son maquetas estáticas. Muestran el diseño visual pero todavía no consumen datos reales del servidor Express.

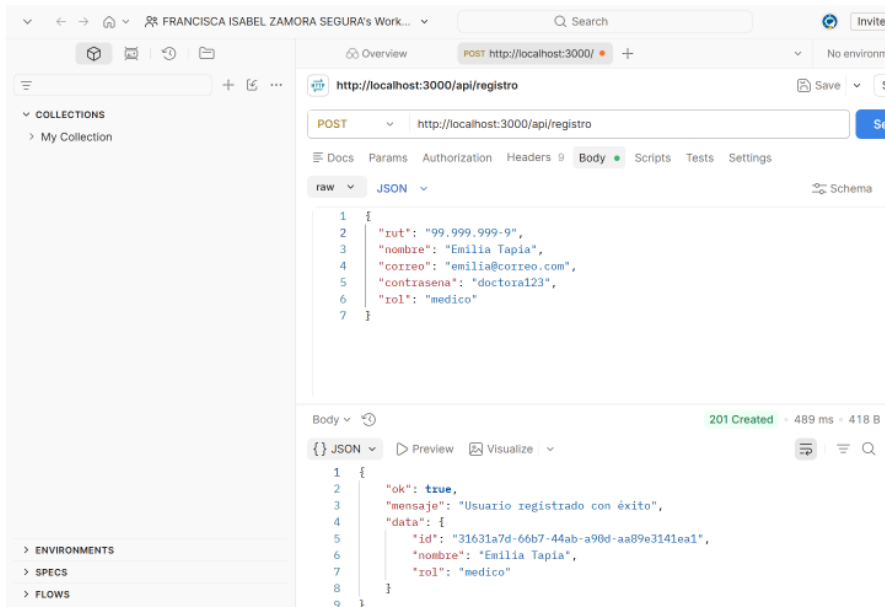
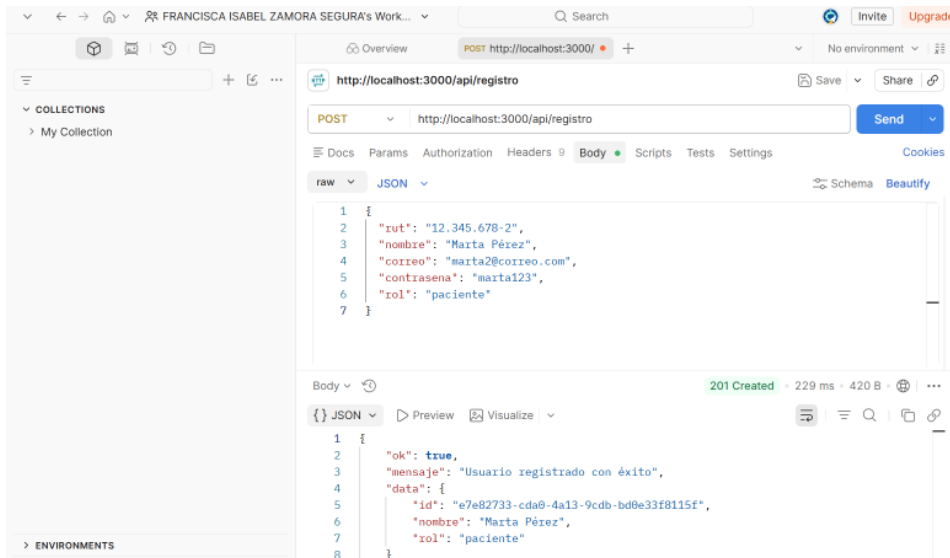
7.Pruebas funcionales (EP 2.7)

Se realizaron pruebas funcionales estructuradas mediante **Postman**. Los resultados y respuestas del servidor proporcionan la evidencia del comportamiento lógico del backend:

Registro de Usuarios Se verificó el flujo de creación de cuentas mediante el envío de solicitudes con payloads JSON estructurados en el cuerpo de la petición. El backend procesa de manera correcta los datos de entrada según los roles del sistema:

- **Rol Paciente:** Al enviar los parámetros de un usuario de prueba (como el RUT 12.345.678-2, nombre Marta Pérez, correo y contraseña), el servidor responde con un estado HTTP 201 Created y un JSON que confirma la persistencia exitosa del registro, retornando un identificador único.

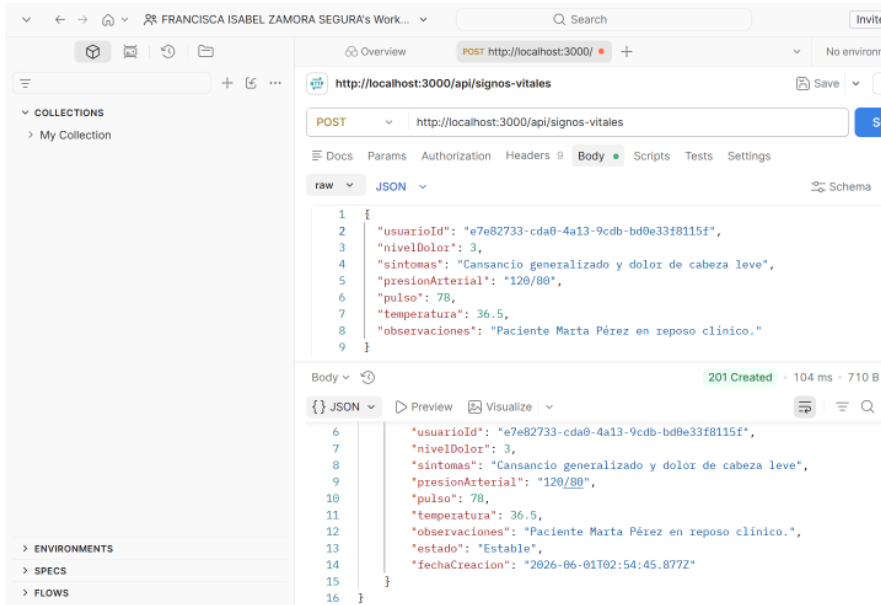
- **Rol Médico:** Se validó la flexibilidad del endpoint al registrar cuentas para el personal, obteniendo en ambos casos respuestas satisfactorias 201 Created acompañadas de sus respectivos IDs autogenerated.



campo estado: "Estable" y guarda el documento clínico reflejando la fecha exacta de creación en el servidor.

- **Evaluación de Parámetros Alterados:** Se realizaron pruebas con registros que presentaban sintomatología severa, confirmando que el algoritmo interno del controlador altera correctamente la propiedad a estado: "Crítico" antes de alojar la fila en MySQL.

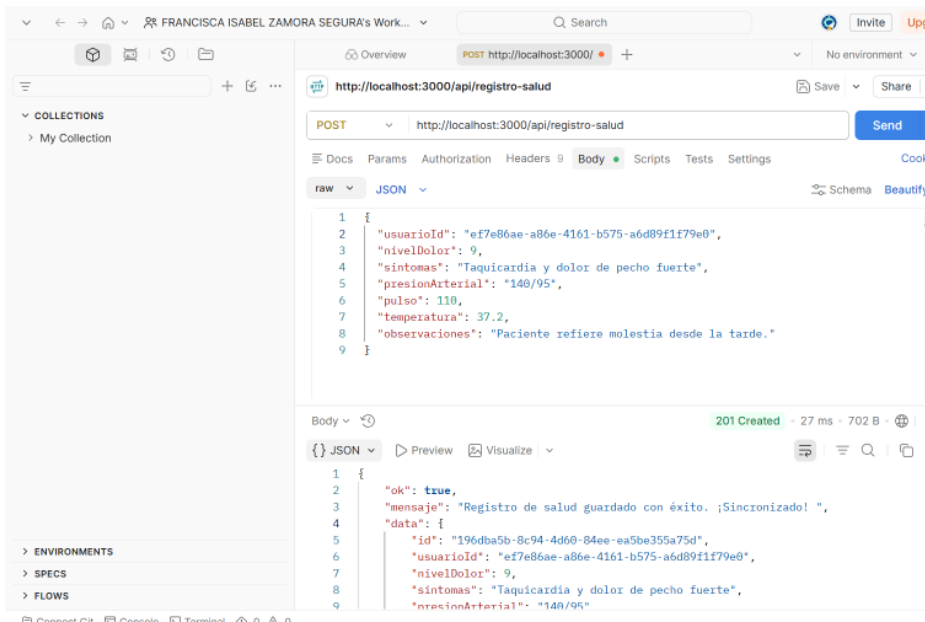
Las respuestas devueltas en formato JSON demuestran el cumplimiento normativo de la API REST respecto al control de errores, la gestión de esquemas de datos y la consistencia en el almacenamiento relacional del ecosistema de telemedicina.



```
1 {
2   "usuarioId": "e7e82733-cda0-4a13-9cdb-bd0e33f8115f",
3   "nivelDolor": 3,
4   "sintomas": "Cansancio generalizado y dolor de cabeza leve",
5   "presionArterial": "120/80",
6   "pulso": 78,
7   "temperatura": 36.5,
8   "observaciones": "Paciente Marta Pérez en reposo clínico."
9 }
```

Body ▾ 201 Created · 104 ms · 710 B

```
{
  "usuarioId": "e7e82733-cda0-4a13-9cdb-bd0e33f8115f",
  "nivelDolor": 3,
  "sintomas": "Cansancio generalizado y dolor de cabeza leve",
  "presionArterial": "120/80",
  "pulso": 78,
  "temperatura": 36.5,
  "observaciones": "Paciente Marta Pérez en reposo clínico.",
  "estado": "Estable",
  "fechaCreacion": "2026-06-01T02:54:45.877Z"
}
```



```
1 {
2   "ok": true,
3   "mensaje": "Registro de salud guardado con éxito. ¡Sincronizado!",
4   "data": {
5     "id": "196dba5b-0c94-4d60-04ee-ea5be35a75d",
6     "usuarioId": "ef7e86ae-a86e-4161-b575-a6d89f1f79e0",
7     "nivelDolor": 9,
8     "sintomas": "Taquicardia y dolor de pecho fuerte",
9     "presionArterial": "140/95"
10  }
11 }
```

Body ▾ 201 Created · 27 ms · 702 B

```
{
  "ok": true,
  "mensaje": "Registro de salud guardado con éxito. ¡Sincronizado!",
  "data": {
    "id": "196dba5b-0c94-4d60-04ee-ea5be35a75d",
    "usuarioId": "ef7e86ae-a86e-4161-b575-a6d89f1f79e0",
    "nivelDolor": 9,
    "sintomas": "Taquicardia y dolor de pecho fuerte",
    "presionArterial": "140/95"
  }
}
```

FRANCISCA ISABEL ZAMORA SEGURA's Work... Search

Overview POST http://localhost:3000/ + No environment

http://localhost:3000/api/registro-salud Save Share

POST http://localhost:3000/api/registro-salud Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "usuarioId": "b147b1d6-76a6-40b8-9701-583220347841",
3   "nivelDolor": 2,
4   "síntomas": "Leve dolor de cabeza",
5   "presiónArterial": "120/80",
6   "pulso": 75,
7   "temperatura": 36.5,
8   "observaciones": "Se siente bien en general."
9 }
```

Body 201 Created · 13 ms · 670 B

{ JSON Preview Visualize

```
1 {
2   "ok": true,
3   "mensaje": "Registro de salud guardado con éxito. ¡Sincronizado!",
4   "data": {
5     "id": "58cccd41-316c-46f7-971f-7ecf4a73ef9d",
6     "usuarioId": "b147b1d6-76a6-40b8-9701-583220347841",
7     "nivelDolor": 2,
8     "síntomas": "Leve dolor de cabeza",
9     "presiónArterial": "120/80"
```

Connect Git Console Terminal 0 0

Entrega final:

1. Implementación de Funcionalidades Completas (EF1)

En esta etapa final se completó el ciclo CRUD en todos los módulos del sistema, conectando el frontend Ionic con el backend Express de manera integral:

Módulo de Chat en Tiempo Real: Se implementó una tabla Mensaje en MySQL con campos usuariold, remitente_tipo, texto y fecha. El sistema permite la comunicación bidireccional entre paciente y médico mediante polling automático cada 10 segundos. El médico dispone de una vista ChatMedico que lista todos sus pacientes asignados con su último mensaje, permitiendo seleccionar una conversación y responder en tiempo real.

Ficha Clínica Completa: Se desarrolló la página FichaPaciente.tsx que consume el endpoint GET /api/registro-salud/:id para mostrar los signos vitales reales del último registro clínico (presión arterial, pulso, temperatura y nivel de dolor), junto con el historial completo de registros anteriores. La ficha incluye datos personales del paciente (nombre, RUT, correo, región y comuna) obtenidos desde GET /api/usuario/:id.

Gestión de Pacientes: Se creó el módulo BuscarPacientes.tsx con búsqueda en tiempo real por nombre o RUT, y ResumenPacientes.tsx con lista de pacientes asignados al médico con semaforización visual por estado. Ambas pantallas permiten acceder a la ficha clínica individual con un clic.

Gestión de Recetas Médicas con CRUD Completo: La vista RecetasMedicas.tsx fue actualizada para diferenciarse según el rol. El médico puede crear nuevas recetas seleccionando un paciente de su lista, ingresando medicamentos e indicaciones. Tanto el médico como el paciente pueden eliminar recetas mediante DELETE /api/recetas/:id.

Videollamada Simulada: Se implementó VideollamadaPaciente.tsx que permite al paciente ver sus citas programadas y unirse a una llamada simulada con interfaz de video, controles de micrófono, cámara y botón de colgar.

Asignación Automática de Médico: Al registrarse un nuevo paciente, el sistema busca automáticamente el primer médico disponible en la base de datos y asigna el medicold correspondiente, garantizando que todos los pacientes queden vinculados a un especialista desde el momento del registro.

2. Mejoras en UI/UX (EF2)

Se realizaron mejoras significativas en la experiencia de usuario en respuesta a los comentarios recibidos en entregas anteriores:

Elementos Clickables: Todos los ítems del menú lateral (Nav.tsx para médico y MenuNavegacion.tsx para paciente) fueron conectados a sus rutas correspondientes mediante history.push() e useOnRouter().

Redirección Automática por Rol: Se corrigió App.tsx para que un usuario ya autenticado no pueda acceder a /login o /registro. El sistema detecta el rol almacenado y redirige automáticamente al dashboard correspondiente (/medico-dashboard o /home).

Diferenciación Visual por Rol: El menú lateral del médico (Nav.tsx) muestra opciones exclusivas como "Buscar Pacientes", "Resumen Pacientes", "Citas" y "Chat Médico", mientras que el menú del paciente (MenuNavegacion.tsx) presenta "Registrar Estado", "Indicaciones", "Recetas", "Exámenes" y "Seguimiento".

Alertas Visuales por Estado: La ficha clínica del paciente muestra una alerta roja destacada cuando el estado es Crítico, y los listados de pacientes utilizan círculos de color (rojo para Crítico, verde para Estable) para una identificación inmediata.

3. Seguridad Avanzada (EF3)

Se implementaron múltiples capas de seguridad adicionales:

Helmet.js: Se incorporó el middleware helmet() en el servidor Express, que configura automáticamente cabeceras HTTP de seguridad para proteger contra ataques XSS, clickjacking y otras vulnerabilidades comunes.

Rate Limiting: Se configuraron dos niveles de limitación de peticiones mediante express-rate-limit. El límite general permite 100 requests por IP cada 15 minutos, mientras que el endpoint /api/login tiene un límite más estricto de 10 intentos por 15 minutos para prevenir ataques de fuerza bruta.

CORS Seguro: La configuración CORS fue actualizada para aceptar peticiones desde localhost y dominios de GitHub Codespaces, rechazando cualquier origen externo no autorizado.

Las contraseñas se almacenan con hash bcrypt mediante bcrypt.genSalt(10) y bcrypt.hash() antes de persistirse en MySQL, garantizando que nunca se guarden en texto plano.

4. Optimización de Consultas (EF4)

Se agregaron índices de base de datos en el archivo schema.prisma para optimizar las consultas más frecuentes del sistema:

```
prisma
```

```
model RegistroSalud {
```

```
  @@index([usuarioid])
```

```
  @@index([fechaCreacion])
```

```
}
```

```
model Cita {
```

```
  @@index([usuarioid])
```

```

    @@index([medicoid])
}

model Mensaje {
    @@index([usuarioid])
}

```

Estos índices aceleran las consultas de historial clínico por paciente, búsqueda de citas por médico y carga del historial de mensajes, que son las operaciones de lectura más frecuentes en el sistema.

5. Integración con Servicio Externo (EF5)

Se integró **Nodemailer** con el servicio de Gmail como sistema de notificaciones automáticas por correo electrónico. El servicio se implementó en `src/servicios/emailService.ts` y se activa automáticamente desde el controlador `registroSaludController.ts` cuando el algoritmo de evaluación clínica determina que el estado de un paciente es **Crítico** (temperatura $\geq 38^{\circ}\text{C}$, pulso ≥ 100 bpm o nivel de dolor ≥ 8).

El email enviado al médico asignado incluye el nombre del paciente, síntomas reportados y la fecha del registro, permitiendo una respuesta oportuna ante situaciones de riesgo clínico.

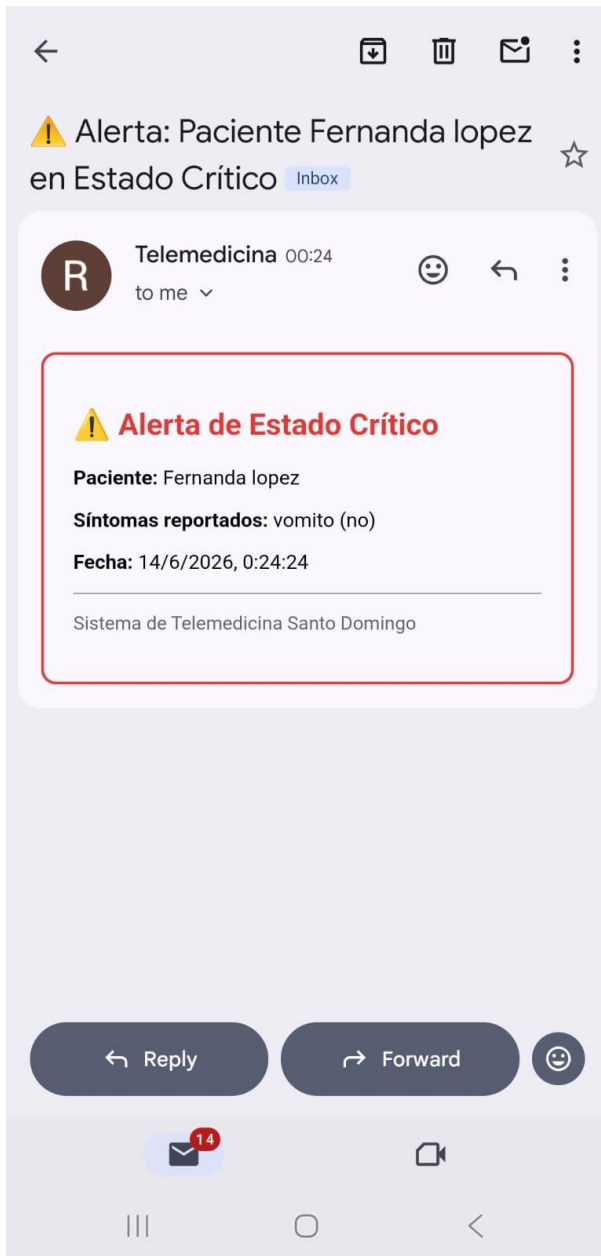
La configuración utiliza variables de entorno para proteger las credenciales:

EMAIL_USER=correo@gmail.com

EMAIL_PASS=contraseña_de_aplicación_gmail

Ejemplo de llegada email:

The screenshot shows a web application running on localhost:8102. The interface has a teal header with 'Contacto' and 'Ayuda' links. A left sidebar contains navigation options: 'Registrar estado de salud' (highlighted in green), 'Inicio', 'Citas', 'Agendar', and a 'Servicios' section with 'Indicación', 'Recetas médicas', 'Exámenes', and 'Seguimiento indicaciones'. The main content area is a form for 'registro-estado'. It includes fields for 'Dolor:' (set to 'Fuerte'), 'Síntomas:' (set to 'Vomito'), 'Presión Arterial:' (120), 'Pulso:' (12), '¿Alguna molestia?:' (set to 'no'), and 'Temperatura:' (39.5). There is also an 'Observaciones:' text area set to 'no'. A green bar is visible at the bottom of the form area. A Windows watermark 'Activar Windows' is present in the bottom right corner.



6. Despliegue con Docker (EF6)

Se configuró el entorno de despliegue mediante Docker para permitir la ejecución del sistema completo sin dependencias externas:

Dockerfile Backend (nodejs-Telemedicina/Dockerfile): Basado en node:22, instala dependencias, genera el cliente Prisma y expone el puerto 3000.

Dockerfile Frontend (Ionic-Telemedicina/Dockerfile): Basado en node:22-alpine, instala dependencias con --legacy-peer-deps y levanta el servidor Vite en el puerto 5173 (mapeado al 8100).

docker-compose.yml: Hace tres servicios: database (MySQL 8.0), backend (Node.js + Express) y frontend (Ionic React), con volumen persistente para la base de datos y dependencias entre servicios para garantizar el orden de inicialización correcto.

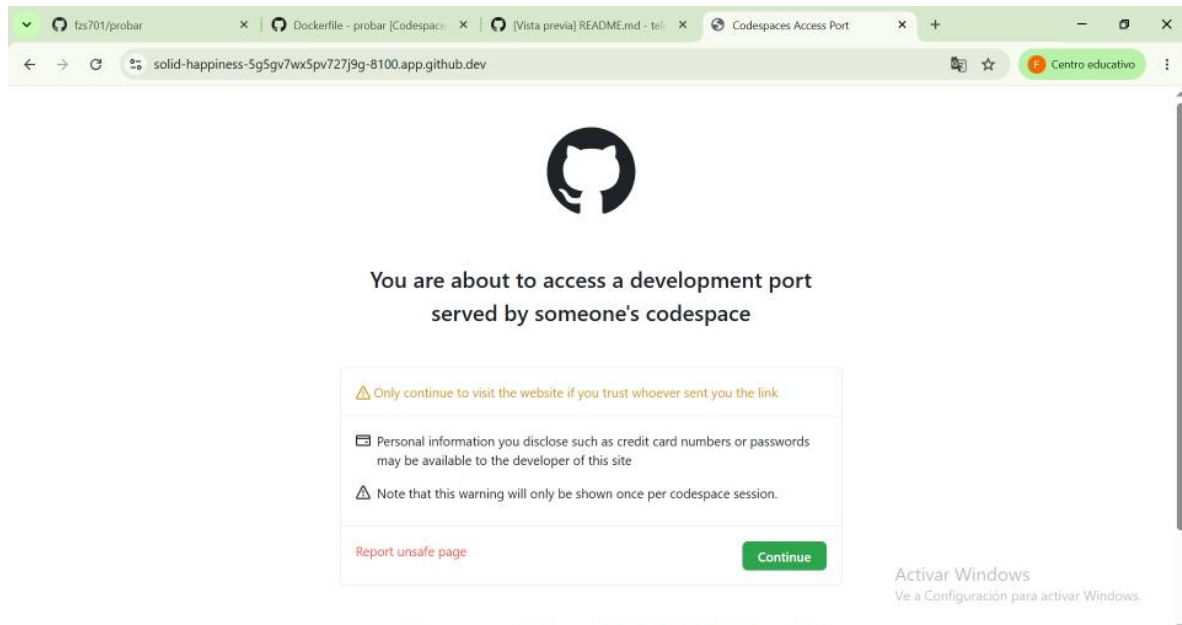
Las pruebas de despliegue se realizaron mediante **GitHub Codespaces**, ya que el equipo local presentó limitaciones de virtualización (Hyper-V no disponible en la BIOS) que impidieron ejecutar Docker Desktop.

En **GitHub Codespaces**, que es un entorno de desarrollo en la nube que permite ejecutar Docker sin restricciones de hardware local. Los pasos seguidos fueron:

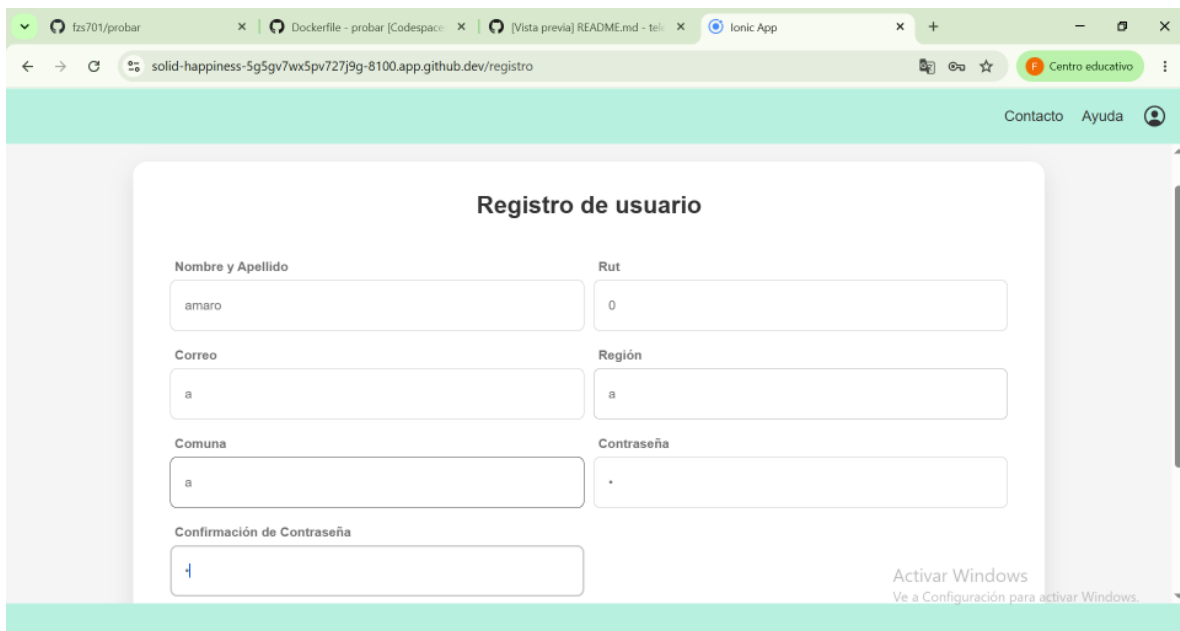
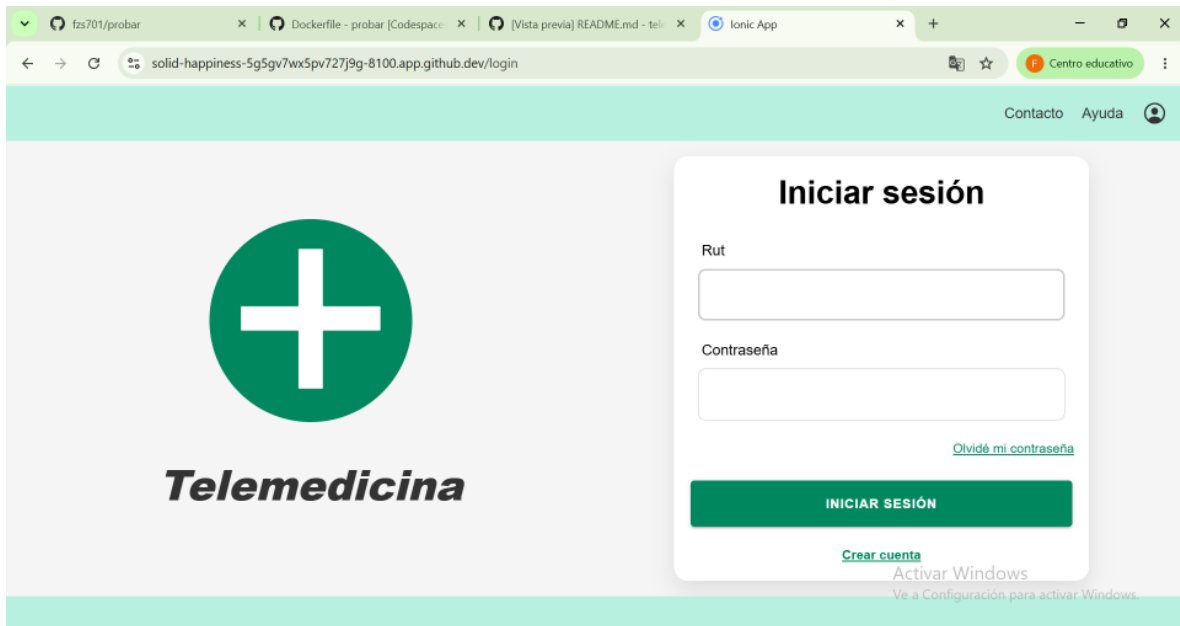
1. Ir al repositorio en GitHub
2. Hacer clic en **Code** → **Codespaces** → **Create codespace on master**
3. Una vez abierto el entorno, abrir la terminal y ejecutar: `docker compose up --build`
4. Esperar a que los tres servicios levanten correctamente
5. En la pestaña **Puertos**, hacer clic derecho sobre el puerto **3000** y cambiar visibilidad a **Public**
6. Acceder al frontend desde el puerto **8100** expuesto por Codespaces

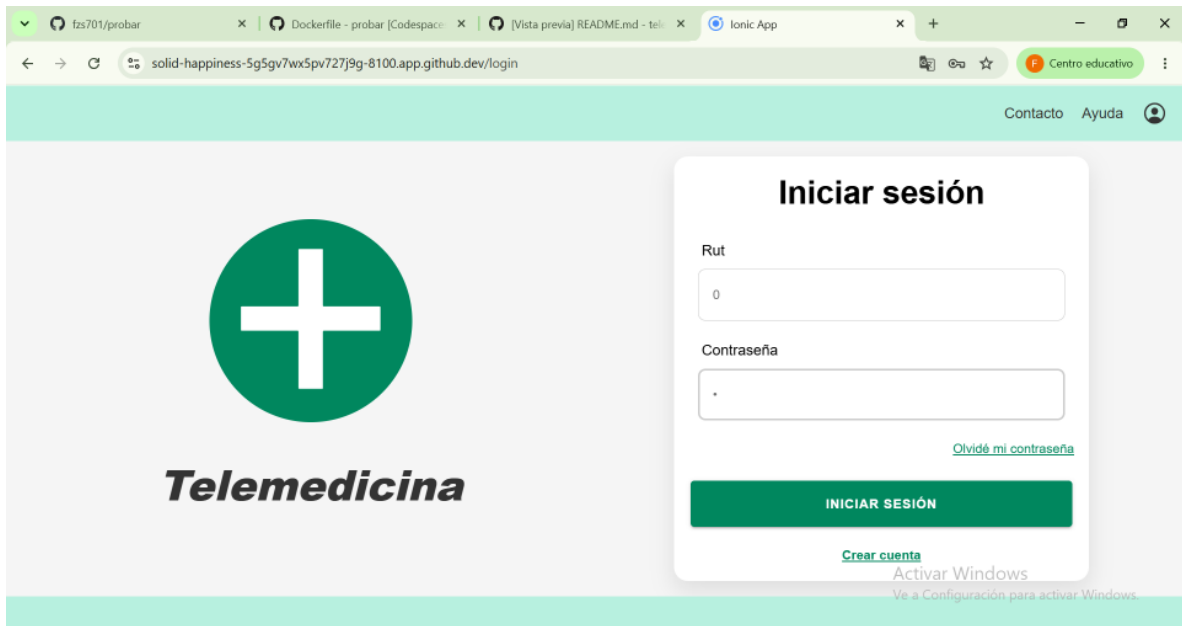
Esto levantó exitosamente los tres servicios, confirmando el correcto funcionamiento del sistema en contenedores Docker, tal como se evidencia en las capturas adjuntas.

1. Se debe apretar continue para verificar

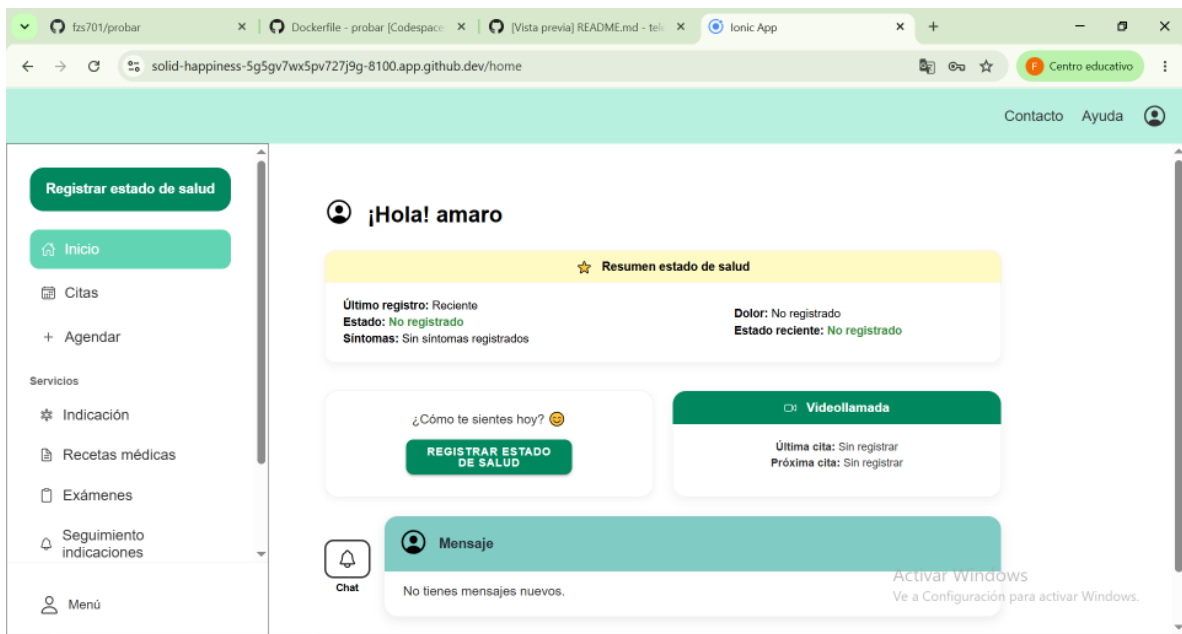


2. Abre exitosamente en navegador.

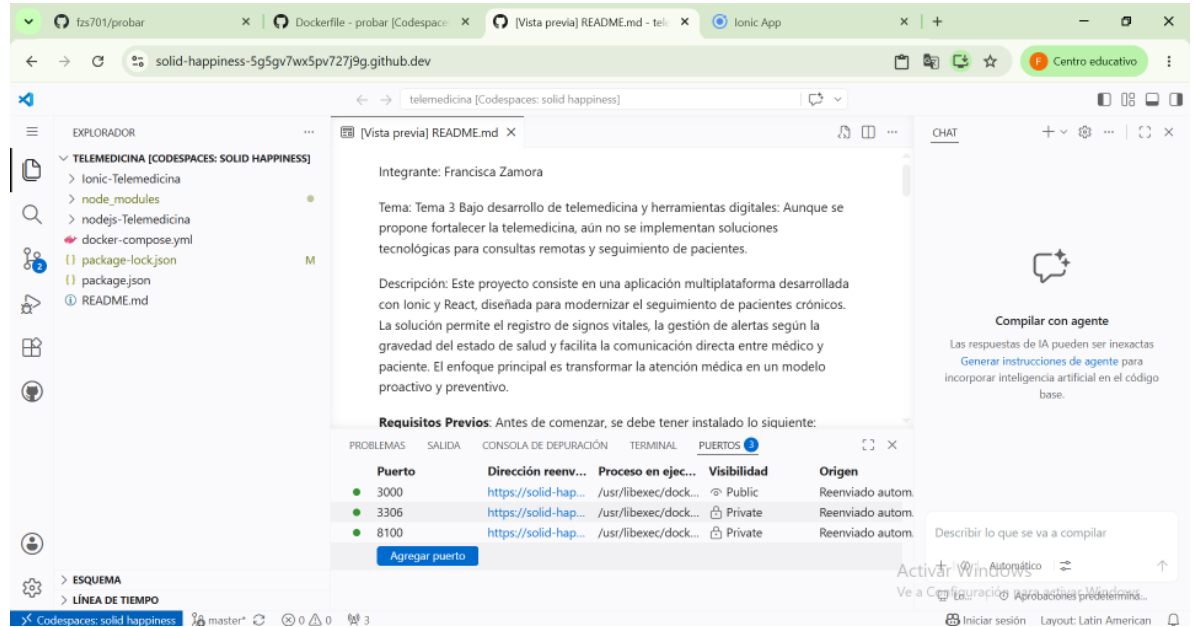




4. Pruebas de que funciona y nos deja entrar.



5. Configuración de puertos en Codespaces: el puerto 3000 (backend) debe estar en Public para que el frontend pueda comunicarse con él desde el navegador.



The screenshot shows a Codespaces environment for a project named 'telemedicina'. The file explorer on the left lists files like 'package-lock.json', 'package.json', and 'README.md'. The central editor area displays the 'README.md' file, which contains information about the project, including the integrator 'Francisca Zamora', the topic 'Tema 3 Bajo desarrollo de telemedicina y herramientas digitales', and a description of the project. The right sidebar features a chat window and a 'Compilar con agente' section. At the bottom, the 'PUERTOS' (Ports) panel is open, showing a table of port mappings.

Puerto	Dirección reenv...	Proceso en ejec...	Visibilidad	Origen
3000	https://solid-hap...	/usr/libexec/dock...	Public	Reenviado autom.
3306	https://solid-hap...	/usr/libexec/dock...	Private	Reenviado autom.
8100	https://solid-hap...	/usr/libexec/dock...	Private	Reenviado autom.

Below the table is a button labeled 'Agregar puerto'.

Link GitHub: <https://github.com/fzs701/telemedicina.git>

Conclusión

La entrega final consolida una plataforma de telemedicina funcional y segura que cumple con todos los requerimientos funcionales definidos en la Entrega Parcial 1. El sistema permite el monitoreo continuo de pacientes, la comunicación en tiempo real entre médico y paciente, la gestión completa de fichas clínicas y la emisión automática de alertas críticas por email. La arquitectura modular implementada, combinada con las capas de seguridad avanzada y el despliegue containerizado con Docker, posiciona al sistema como una solución escalable y mantenible para la atención médica remota.